

온디바이스 ONNX Runtime sLLM 추론의 Decode 병목 분석과 FPGA 기반 INT8 MatVec 가속기 구조 제안

Decode Bottleneck Analysis of On-device ONNX Runtime sLLM Inference and an FPGA-based INT8 MatVec Accelerator Architecture Proposal

최윤희

한국디지털미디어고등학교

Yunhyuk Choi

Korea Digital Media High School

초록

온디바이스 소형 언어모델(sLLM) 추론에서는 모델 크기뿐 아니라 ONNX graph 구조, execution provider, quantization 상태, decode cache 처리 방식, host/offload interface가 token 단위 실행 비용을 결정한다. 본 연구는 Gemma 계열 sLLM의 ONNX Runtime profiling 결과와 Lenovo Y700(TB320FC, Snapdragon 8+ Gen 1급 taro platform)에서 실행한 representative projection micrograph 실측을 결합해 decode 단계의 projection-heavy 병목을 분석하고, 그 결과를 FPGA 기반 INT8 MatVec 구조 요구 사항으로 정리한다. 기존 ONNX Runtime CPUExecutionProvider profiling에서는 MatMul이 decode trace node 시간의 81.1%를 차지했으며, MatMul 내부에서는 mlp_projection과 lm_head가 88.90%를 차지했다. Y700의 ONNX Runtime 1.27.0 Android APK 실험에서 INT8 MatMulInteger p50 latency는 CPU EP 기준 attention output 0.738 ms, lm_head tile 3.582 ms, MLP projection 3.428 ms였고, NNAPI EP에서는 각각 0.518 ms, 2.989 ms, 3.333 ms였다. 추가로 QAI RT 2.47.0의 qnn-net-run direct DLC 경로를 사용해 Y700 QNN HTP 실행을 확인했으며, float lm_head tile의 10회 평균 NetRun execute는 3.415 ms, accelerator execute는 1.668 ms였다. 반면 MLP projection에서는 HTP 평균 NetRun execute가 19.536 ms로 측정되어, QNN/HTP 경로에서도 shape와 weight movement가 성능을 지배할 수 있음을 보였다. DE10-Lite에서는 16x4 INT8 MatVec 연산 단위의 board-level correctness를 확인했으며, 20회 JTAG-to-Avalon 호출에서 CPU reference와 동일한 결과 및 internal cycle counter 기준 65 cycles, 1.3 us @ 50 MHz를 확보했다. 이 보드 검증은 병목 분석에서 도출한 MatVec 계열 연산을 하드웨어 연산 단위로 구현하고 cycle-level로 관측할 수 있음을 보이는 PoC이다. Roofline/interface 분석은 weight movement가 projection-scale offload의 지배 조건

임을 보이며, 후속 구조는 weight residency와 provider/runtime 호출 경계를 함께 줄이는 memory-centric low-bit MatVec 방향으로 귀결된다.

키워드: ONNX Runtime, 온디바이스 추론, 소형 언어모델, decode, MatMul, MatVec, FPGA, INT8, DE10-Lite

Abstract

On-device small language model inference is shaped not only by model size, but also by ONNX graph structure, execution providers, quantization state, decode-cache handling, and host/offload interfaces. This study analyzes decode-stage projection bottlenecks by combining ONNX Runtime profiling results for Gemma-class sLLM workloads with representative projection micrograph measurements on a Lenovo Y700 Android tablet. In the existing ONNX Runtime CPUExecutionProvider profile, MatMul accounts for 81.1% of traced decode node time, while mlp_projection and lm_head together account for 88.90% of MatMul time. On the Y700, an ONNX Runtime 1.27.0 APK benchmark reports INT8 MatMulInteger p50 latencies of 0.738 ms, 3.582 ms, and 3.428 ms for attention-output, lm_head tile, and MLP projection micrographs on the CPU EP; the corresponding NNAPI EP p50 latencies are 0.518 ms, 2.989 ms, and 3.333 ms. A separate QAIRT 2.47.0 qnn-net-run direct DLC experiment confirms QNN HTP execution on the Y700: for a float lm_head tile, the 10-run mean NetRun execute time is 3.415 ms and the accelerator execute time is 1.668 ms. For the MLP projection, however, the HTP mean NetRun execute time is 19.536 ms, showing that shape and weight movement can still dominate the QNN/HTP path. On the FPGA side, a 16x4 INT8 MatVec compute unit is validated on DE10-Lite with board-level correctness: 20 JTAG-to-Avalon invocations match the CPU reference, and the internal cycle counter reports 65 cycles, or 1.3 us at 50 MHz. This validation is used as a PoC for functional correctness and cycle-level observability of an INT8 MatVec unit, while the roofline/interface analysis shows that projection-scale offload is dominated by weight movement and requires a memory-centric low-bit MatVec path with weight residency and a low-overhead invocation boundary.

Keyword: ONNX Runtime, on-device inference, small language model, decode, MatMul, MatVec, FPGA, INT8, DE10-Lite

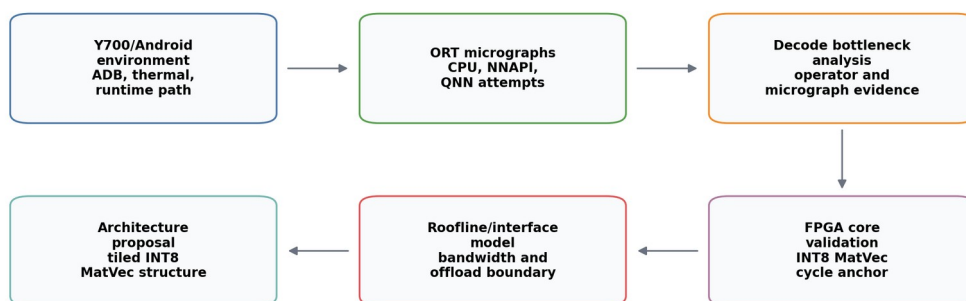
1. 서론

온디바이스 sLLM 추론은 클라우드 의존성을 낮추고 개인정보 보호와 저지연 응답 가능성을 제공하지만, 실제 배포 계층에서는 모델 파라미터 수만으로 병목을 설명하기 어렵다.

Autoregressive language model은 prompt 전체를 처리하는 prefill과 다음 token을 반복 생성하는 decode로 나뉜다. Decode에서는 token dimension이 작아지더라도 hidden dimension, projection dimension, cache tensor의 lifetime은 유지되므로 각 token마다 projection, cache access, graph-level shape operation이 결합된다.

본 연구는 ONNX Runtime에서 관측되는 decode 병목을 분석하고, 이를 FPGA INT8 MatVec/MatMul 구조 요구사항으로 변환하는 것을 목표로 한다. 특히 기존 LLM accelerator 논의가 attention 또는 QK score 중심으로 흐르기 쉬운 점을 고려하여, 실제 ONNX Runtime trace에서 MLP projection과 lm_head가 차지하는 비중을 확인하고, QK-only가 아닌 projection-heavy low-bit MatVec/MatMul 경로의 필요성을 검토한다.

본 연구의 기여는 세 가지이다. 첫째, ONNX export, graph inspection, runtime profiling, Android/Y700 실행 하네스, FPGA 연산 단위 검증을 증거 계층별로 분리한다. 둘째, MatMul 중에서도 mlp_projection과 lm_head가 큰 비중을 차지한다는 점을 바탕으로 memory-centric low-bit MatVec/MatMul 구조 요구사항을 도출한다. 셋째, DE10-Lite 16x4 INT8 MatVec 연산 단위의 board-level correctness와 cycle-level validation을 제시하고, 이를 projection-scale 구조 요구사항 및 offload granularity 분석과 연결한다.



Evidence types are kept separate: measured Android results, board-measured FPGA cycles, simulation, and projected models.

그림 1. 전체 연구 흐름

그림 1은 Android/Y700 실행 경로, ONNX Runtime 분석, FPGA core validation, projection-scale roofline/interface model을 서로 다른 증거 계층으로 분리한 연구 흐름을 나타낸다.

2. 관련 연구 및 배경

Transformer 추론에서 prefill은 입력 prompt 전체를 한 번에 처리하고, decode는 cache를 참조하며 token을 순차적으로 생성한다. Decode 단계는 batch와 token dimension이

작아질 수 있지만, 각 layer의 MLP projection, attention projection, lm_head projection은 반복된다. 따라서 decode 병목은 attention score 계산뿐 아니라 dense projection, cache movement, graph-level shape operation을 함께 보아야 한다.

KV-cache는 long-context decode에서 핵심 구조이다. Orca는 autoregressive serving에서 iteration-level scheduling의 중요성을 보였고[10], vLLM/PagedAttention은 KV-cache를 block 단위로 관리하여 memory fragmentation과 scheduling 문제를 줄이는 방향을 제시했다[3]. FlashAttention 계열 연구는 attention kernel의 I/O-aware tiling과 work partitioning을 최적화한다[4][11]. 이러한 연구들은 decode와 memory movement의 중요성을 보여준다. 본 연구는 이 맥락에서 ONNX Runtime 배포 계층의 projection, cache movement, graph-level shape operation을 측정 대상으로 삼는다.

ONNX Runtime은 graph optimization과 execution provider를 통해 같은 ONNX graph라도 CPU, NNAPI, QNN 등 다양한 경로로 실행할 수 있다[9]. 온디바이스 배포에서는 provider 선택, quantization state, graph rewrite가 병목을 크게 바꿀 수 있다. GPTQ, SmoothQuant, AWQ와 같은 quantization 연구는 LLM 배포에서 weight/activation 표현 방식이 실행 비용과 정확도에 함께 영향을 준다는 점을 보여준다[5][6][7]. 본 연구는 현재 확보된 CPUExecutionProvider trace와 Android 실행 하네스를 분리하여, 측정된 값과 아직 실행되지 않은 경로를 혼동하지 않는다.

FPGA 기반 transformer accelerator 연구로는 FTRANS, DFX, FlightLLM 등이 있다[8][12][13]. 이 연구들은 full model mapping, multi-FPGA appliance, complete mapping flow 등 더 큰 시스템 범위를 다룬다. 본 연구는 그 이전 단계의 runtime-hardware boundary에 초점을 둔다. ONNX Runtime profiling에서 projection-heavy primitive를 식별하고, Android provider 경로에서 대표 projection micrograph의 latency를 측정한 뒤, DE10-Lite의 최소 INT8 MatVec core 검증과 roofline model을 통해 후속 FPGA 구조가 만족해야 할 memory/interface 조건을 도출한다.

3. 실험 방법

본 연구의 증거 계층은 표 1과 같이 구분한다. 이 구분은 측정값, projected model, invocation overhead를 같은 성능 순위로 혼동하지 않기 위한 방법론이다.

표 1. 실험 환경 및 증거 계층 요약

환경	evidence type	상태	해석 범위
Lenovo Y700 Android	measured APK micrograph	completed	CPU/NNAPI provider latency
ONNX Runtime CPU profile	measured host profile	기존 profiling artifact	trace node time의 operator share
ONNX micrograph	graph evidence	available	representative

환경	evidence type	상태	해석 범위
manifest			projection shape
DE10-Lite INT8 MatVec	board_measured	pass 20/0	16x4 core correctness 및 cycle-level validation
Projection roofline	projected	model only	bandwidth/ weight-movement estimate
후속 구조 방향	proposal	requirement only	memory-centric low-bit MatVec 방 향

Lenovo Y700 경로에서는 adb로 연결된 TB320FC 장치에서 ONNX Runtime 1.27.0 Android APK를 실행했다. 장치는 Android 15, arm64-v8a ABI, Qualcomm taro platform으로 확인되었으며, /proc/meminfo의 MemTotal은 15,578,208 kB로 기록되었다. APK는 ONNX model을 asset으로 포함하고 session.run wall-clock latency를 warmup 3회, 측정 20회로 기록한다. CPUExecutionProvider와 NNAPIExecutionProvider는 실행되었다. 사용한 ONNX Runtime Android AAR의 available provider 목록은 [CPU, NNAPI, XNNPACK, WEBGPU]였으므로 ORT QNN Execution Provider는 APK 안에서 사용할 수 없었다. 대신 Windows Pocket4의 QAIRT 2.47.0 설치본을 사용해 qnn-onnx-converter, snpe-onnx-to-dlc, Android qnn-net-run direct DLC 경로를 별도로 검증했다.

표 2. Lenovo Y700 ONNX Runtime projection micrograph 결과

micrograph	dtype/op	CPU EP p50	NNAPI EP p50	QNN EP
attention output 1024x1152	INT8 MatMulInteger	0.738 ms	0.518 ms	ORT AAR 미 지원
lm_head tile 1152x4096	INT8 MatMulInteger	3.582 ms	2.989 ms	ORT AAR 미 지원
MLP projection 1152x6912	INT8 MatMulInteger	3.428 ms	3.333 ms	ORT AAR 미 지원
16x4 provider	INT8 MatMulInteger	0.051 ms	0.159 ms	ORT AAR 미 지원

micrograph	dtype/op	CPU EP p50	NNAPI EP p50	QNN EP
overhead	ger			
probe				

QAIRT direct 경로는 ORT QNN EP와 동일한 API 경로가 아니므로 표 2와 직접 합산하지 않는다. MatMulInteger micrograph는 QAIRT ONNX converter의 dry-run에서 MatMulInteger: unsupported in Converter로 보고되었다. 따라서 QNN direct 실행은 float MatMul DLC로 제한하고, weight를 initializer가 아닌 입력 tensor로 두는 기존 micrograph 구조를 유지했다. 이 조건은 QNN HTP 지원 여부와 실행 overhead를 확인하는 데에는 유용하지만, weight-resident deployment model을 대표하지 않는다.

표 2a. Lenovo Y700 QAIRT qnn-net-run direct DLC 결과

graph	backend	runs	backend 또는 NetRun accelerator		비고
			execute mean/p50/ p95	mean/p50/ p95	
lm_head tile 1152x4096 float	QNN CPU	10	14.192 / 12.709 / 26.671 ms	12.032 / 10.750 / 22.504 ms	direct DLC, dynamic weight input
MLP projection 1152x6912 float	QNN CPU	10	16.078 / 15.885 / 19.955 ms	12.399 / 12.245 / 15.958 ms	direct DLC, dynamic weight input
lm_head tile 1152x4096 float	QNN HTP	10	3.415 / 3.166 / 4.253 ms	1.668 / 1.770 / 1.898 ms	accelerator execute, HVX threads 4
MLP projection 1152x6912 float	QNN HTP	10	19.536 / 19.125 / 23.533 ms	13.945 / 13.789 / 15.887 ms	accelerator execute, HVX threads 4

표 2a의 HTP 결과는 lm_head tile에서는 provider/accelerator 경계가 의미 있는 실행 경로가 될 수 있음을 보여준다. 반면 MLP projection에서는 HTP의 accelerator execute 및 RPC/QNN execute 시간이 커져 CPU direct 실행과 비교해 일관된 이득으로 해석하기 어렵

다. 이 차이는 projection workload를 단순히 accelerator에 보내는 것만으로 충분하지 않고, weight residency, input/output movement, graph initialization/finalization, invocation granularity를 함께 다뤄야 함을 뒷받침한다.

대표 micrograph는 표 3과 같이 생성하였다. 파일명은 intended role을 나타낼 수 있으나, 본 논문에서는 graph inspection으로 확인한 operator, dtype, tensor shape를 기준으로 해석한다.

표 3. ONNX micrograph manifest 요약

model	op	input x weight -> output	dtype
matvec_cpu_baseline.onnx	MatMul	1x16 x 16x4 -> 1x4	FLOAT
matvec_int8_matmulinteger.onnx	MatMulInteger	1x16 x 16x4 -> 1x4	INT8 -> INT32
gemma_mlp_projection_1152x6912_float.onnx	MatMul	1x1152 x 1152x6912 -> 1x6912	FLOAT
gemma_mlp_projection_1152x6912_matmulinteger.onnx	MatMulInteger	1x1152 x 1152x6912 -> 1x6912	INT8 -> INT32
gemma_lm_head_tile_1152x4096_float.onnx	MatMul	1x1152 x 1152x4096 -> 1x4096	FLOAT
gemma_lm_head_tile_1152x4096_matmulinteger.onnx	MatMulInteger	1x1152 x 1152x4096 -> 1x4096	INT8 -> INT32
gemma_attention_output_projection_1024x1152_float.onnx	MatMul	1x1024 x 1024x1152 -> 1x1152	FLOAT
gemma_attention_output_projection_1024x1152_matmulinteger.onnx	MatMulInteger	1x1024 x 1024x1152 -> 1x1152	INT8 -> INT32

FPGA 검증은 SpinalHDL 기반 INT8 MatVec core, Verilator simulation, Quartus clean rebuild, DE10-Lite JTAG-to-Avalon register invocation으로 구성된다. JTAG-to-Avalon path는 board-level correctness 확인에 사용하고, 연산 latency는 RTL 내부 cycle counter로 기록한다.

4. ONNX Runtime 및 Micrograph 병목 분석

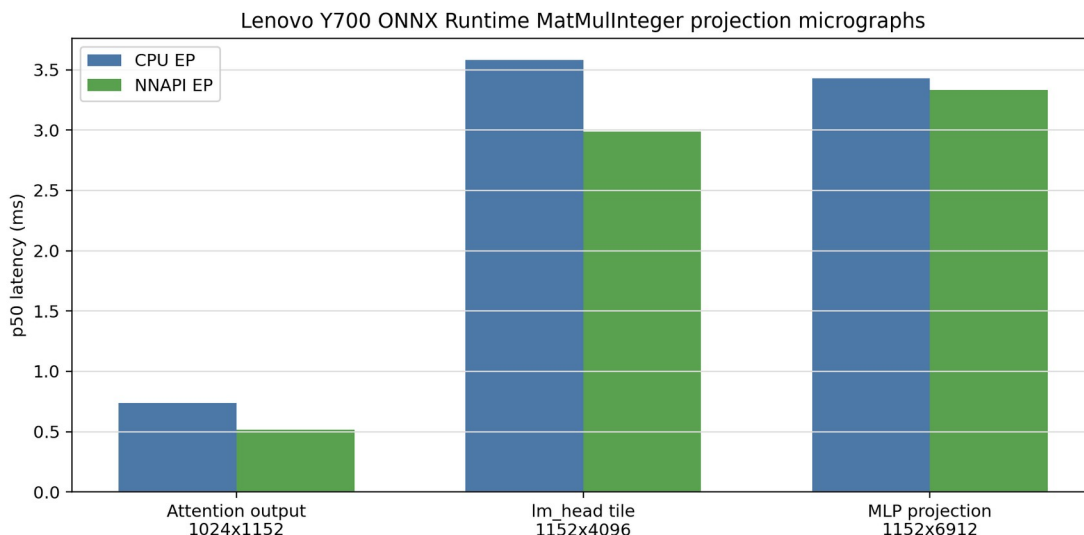
기존 ONNX Runtime CPUExecutionProvider profiling에서는 decode trace node 시간 중 MatMul이 81.1%를 차지했다. Prefill과 decode를 합산한 trace node 시간에서는 MatMul share가 67.5%, prefill에서는 53.4%였다. Long-decode trace에서도 MatMul은 주요 operator group으로 유지되지만, context 2048과 decode 256 조건에서는 Expand + Concat + Unsqueeze 합산 비중이 17.71%까지 증가했다. 이 결과는 decode 병목이 dense projection만으로도, KV-cache만으로도 완전히 설명되지 않으며 두 계층을 함께 다뤄야 함을 의미한다.

표 4. ONNX Runtime profiling 기반 decode 병목 요약

측정 범위	주요 결과	해석
decode MatMul share	81.1%	CPUExecutionProvider trace node time 기준
prefill+decode MatMul share	67.5%	host CPU profiling artifact
mlp_projection + lm_head share	88.90% of MatMul	projection-heavy workload
context 2048, decode 256 shape/cache ops	17.71%	Expand + Concat + Unsqueeze, exploratory trace

MatMul category 분석에서는 mlp_projection과 lm_head가 전체 MatMul 시간의 88.90%를 차지했다. 따라서 후속 FPGA 구조 요구사항은 QK dot-product 전용 block보다 MLP, attention projection, lm_head에 공통 적용 가능한 low-bit MatVec/MatMul 처리를 우선 고려해야 한다. attention_qk_score가 runtime classifier에서 0.00%로 나타난 것은 QK 연산 부재가 아니라, ONNX Runtime graph optimization 과정에서 MatMul, Add, Softmax가 attention 또는 fused operator 내부로 흡수되었거나 현재 event classifier에서 확정 가능한 MatMul event로 분류되지 않았음을 뜻한다. 그러므로 QK 비용의 세부 분해는 kernel-level profiling 또는 optimization-disabled trace가 필요하다.

Long-decode sweep의 일부 결과는 runs 1, warmup 0 조건으로 수집된 exploratory trace이다. 그러므로 latency benchmark로 해석하지 않고 operator share 경향으로만 사용한다. 최종 온디바이스 latency 판단은 Y700 APK micrograph benchmark를 우선한다.



Android APK, ONNX Runtime 1.27.0, warmup=3, runs=20. QNN EP was not available in this AAR build.

그림 2. Lenovo Y700 ONNX Runtime INT8 projection micrograph 결과

그림 2는 INT8 MatMulInteger projection micrograph의 p50 latency를 CPU EP와 NNAPI EP로 나누어 나타낸다. 16x4 graph에서는 provider dispatch overhead가 지배적이므로 구조 비교에 쓰지 않고, 1024~6912 output dimension의 projection micrograph를 온디바이스 projection 연산의 절대 latency와 provider granularity를 살피는 근거로 사용한다. QAI RT direct QNN 실험은 이 결과를 대체하지 않고, QNN HTP 경로가 graph shape와 data movement에 민감하다는 추가 근거로만 사용한다. 전체 Gemma graph 대신 micrograph를 사용한 이유는 Android 환경의 full export, dynamic KV-cache shape, graph copy 비용을 한꺼번에 섞지 않고 dense projection 연산의 provider별 dispatch와 compute 비용을 격리하기 위해서이다.

5. 병목 분석 기반 FPGA 구조 요구사항

기존 ONNX Runtime profiling은 decode trace 안에서 projection-heavy MatMul의 비중을 보여주고, Y700 micrograph는 해당 projection shape가 Android provider 경로에서 어느 정도의 절대 latency를 갖는지 보여준다. 두 증거는 같은 현상을 직접 동일하게 측정한 것이 아니라, 병목 후보와 온디바이스 실행 비용을 서로 다른 계층에서 보완한다. 따라서 후속 FPGA 구조는 단순히 MAC core cycle을 줄이는 방향보다 weight movement, activation reuse, partial sum reuse, output tile 처리, provider/runtime 호출 경계의 비용을 함께 줄이는 memory-centric low-bit MatVec 구조가 되어야 한다.

5.1 Projection workload와 FPGA 구성요소의 대응

본 연구에서 도출한 FPGA 구조 요구사항은 단순히 MatMul 비중이 높다는 사실에서 바로 도출되지 않는다. Decode 단계의 projection workload는 token activation vector가 상

대적으로 작고, 반복적으로 참조되는 weight matrix가 크다는 데이터 특성을 갖는다. 따라서 FPGA offload가 의미를 가지려면 연산 코어 자체보다 weight movement, activation reuse, output tile 처리, invocation granularity가 함께 설계되어야 한다. Y700 micrograph에서 16x4 graph는 NNAPI EP가 CPU EP보다 느렸지만, 1024~6912 output dimension을 갖는 projection graph에서는 NNAPI EP가 유리하거나 유사한 결과를 보였다. QAIRT HTP direct 실행에서도 lm_head tile은 accelerator execute가 1.668 ms로 관측된 반면, 더 큰 MLP projection은 NetRun execute 평균이 19.536 ms로 증가했다. 이는 작은 연산을 잘게 offload하는 방식보다 projection tile 단위의 충분히 큰 granularity와 weight-resident data path를 함께 갖는 하드웨어 경계가 필요함을 시사한다.

표 5. ONNX/Decode 병목과 FPGA 구조 요소의 대응

ONNX/Decode 병목	데이터 특성	FPGA 구조 요구사항	이번 논문 근거
MLP projection	token activation은 작고 1152x6912 weight tile이 큼	multi-lane INT8 또는 low-bit MatVec datapath, weight tile memory, INT32 partial sum buffer	ORT MatMul category, Y700 MLP micrograph, QNN HTP direct MLP 결과
lm_head tile	output dimension이 크고 weight movement가 지배적	weight-resident memory pool, output tile buffer, top-k 또는 host-side reduction boundary	ORT lm_head share, Y700 1152x4096 tile, QNN HTP direct lm_head 결과
attention output projection	attention 이후 dense projection 반복	shared MatVec datapath, activation scratchpad, output tile writeback	Y700 1024x1152 projection micrograph
16x4 provider overhead probe	작은 graph에서는 provider dispatch overhead가 상대적으로 큼	projection-scale offload granularity, low-overhead invocation boundary	Y700 16x4 CPU/NNAPI 비교
long-decode	context 증가 시	accelerator와 별도	long-decode

ONNX/Decode 병목	데이터 특성	FPGA 구조 요구사항	이번 논문 근거
shape/cache ops	Expand + Concat + Unsqueeze 비중 증가	로 graph/runtime specialization, cache metadata 처리 경계 필요	exploratory trace

후속 산출물에서는 이 요구사항을 바탕으로 더 낮은 bit-width와 weight-resident memory 구조를 검토할 수 있다. 1.58bit 계열 모델 변환과 MatMul-free language modeling은 곱셈기 중심 datapath를 줄이는 방향의 참고점이지만[1][2], 본 논문의 INT8 MatVec 검증이 해당 ternary/adder datapath를 직접 검증하는 것은 아니다.

DDR2/LPDDR2 다채널 memory, SRAM-like scratchpad FPGA, compute FPGA 간 custom connector는 이후 특기자 산출물에서 별도 설계와 검증이 필요한 후보로 둔다.

6. DE10-Lite INT8 MatVec primitive 검증 결과

현재 board-measured 결과는 DE10-Lite의 fixed 16x4 INT8 MatVec primitive에 한정된다. 해당 core는 64 MAC PoC workload를 수행하며, 새 Verilog mirror를 Windows Pocket4의 Quartus 25.1std Lite에서 clean compile한 뒤 DE10-Lite에 programming 하여 20회 JTAG-to-Avalon invocation을 수행했다. 결과는 pass_count=20, fail_count=0이며 CPU reference와 동일한 result vector를 기록했다. Internal cycle counter는 65 cycles, 50 MHz 기준 1.3 us를 보고했다.

이 결과는 INT8 MatVec 연산 단위가 실제 보드에서 올바르게 동작하고 내부 cycle counter가 예상 범위의 cycle을 보고한다는 기능 정합성 및 cycle-level validation이다. JTAG-to-Avalon 경로는 기능 검증용 host-tool invocation이며, 성능 해석에는 RTL 내부 cycle counter 값을 사용한다.

16x4 PoC는 lm_head나 MLP projection의 latency를 대표하지 않는다. 그러나 이 검증은 후속 구조 설계에 필요한 최소 하드웨어 경로를 확인한다는 점에서 의미가 있다. 첫째, host가 register interface를 통해 연산 단위를 설정하고 시작하며 결과를 읽는 control path를 구성했다. 둘째, INT8 input과 weight를 INT32 accumulation 결과로 변환하는 fixed-point MatVec datapath가 CPU reference와 bit-exact하게 일치함을 확인했다. 셋째, 외부 invocation overhead와 분리된 internal cycle counter를 통해 core 내부 연산 구간을 관측할 수 있음을 확인했다. 후속 projection-scale 구조는 이 PoC를 단순히 차원만 키우는 방식이 아니라, multi-lane datapath, weight-resident tile memory, activation scratchpad, output tile buffer, low-overhead streaming interface를 추가하는 방향으로 확장되어야 한다.

표 6. DE10-Lite board validation summary

항목	값
input/output dimension	16 / 4
MACs	64
pass_count / fail_count	20 / 0
compute cycles	65
compute time	1.3 us @ 50 MHz
logic elements	2,560 / 49,760
DSP 9-bit elements	1 / 288
memory bits	512 / 1,677,312
Fmax	56.670 MHz

표 7. DE10-Lite PoC와 후속 projection-scale 구조의 차이

항목	현재 DE10-Lite PoC	후속 구조 요구
연산 크기	16x4, 64 MAC	MLP, attention output, lm_head projection tile
datapath	단일 INT8 MatVec core	multi-lane INT8 또는 low-bit MatVec datapath
weight	register-level test vector	weight tile memory 또는 weight-resident memory pool
activation	small fixed input vector	activation scratchpad와 token-level reuse
output	4-element INT32 result	INT32 partial sum buffer와 output tile buffer
interface	JTAG-to-Avalon register invocation	DMA, streaming, shared-memory class interface
검증 의미	bit-exact reference comparison, internal cycle counter	projection-scale throughput, bandwidth, offload granularity 검증

7. Memory/Interface 요구사항과 후속 구조 방향

Projection-scale model은 weight movement와 interface bandwidth가 실제 offload 가능성을 지배할 수 있음을 보여준다. 본 논문에서는 다음 세 식으로 계산 범위를 제한한다.

$$T_{\text{compute}} = \text{MACs} / (\text{lanes} \times f_{\text{clk}})$$

$$T_{\text{stream}} = W_{\text{bytes}} / B_{\text{interface}}$$

$$B_{\text{required}} = W_{\text{bytes}} / T_{\text{target}}$$

여기서 lanes와 f_clk는 FPGA 연산 병렬도와 clock, W_bytes는 token당 weight movement, B_interface는 외부 interface bandwidth 가정이다. 예를 들어 full lm_head 1152->262144 projection은 token당 약 3.02억 MAC과 약 302 MB의 INT8 weight movement를 요구한다. 16 lanes, 50 MHz 가정에서 compute estimate는 약 377 ms이고, 외부 streaming prototype을 상정한 USB3 320 MB/s model case의 stream estimate는 약 947 ms이다. 이 값은 Y700 lm_head tile NNAPI p50 2.989 ms와 측정 조건이 다른 손익분기 분석용 model이다. $T_{\text{stream}} > T_{\text{compute}}$ 조건에서는 interface와 weight residency가 병목이므로, 실사용 offload는 weight-resident memory와 low-overhead invocation boundary를 함께 만족해야 한다.

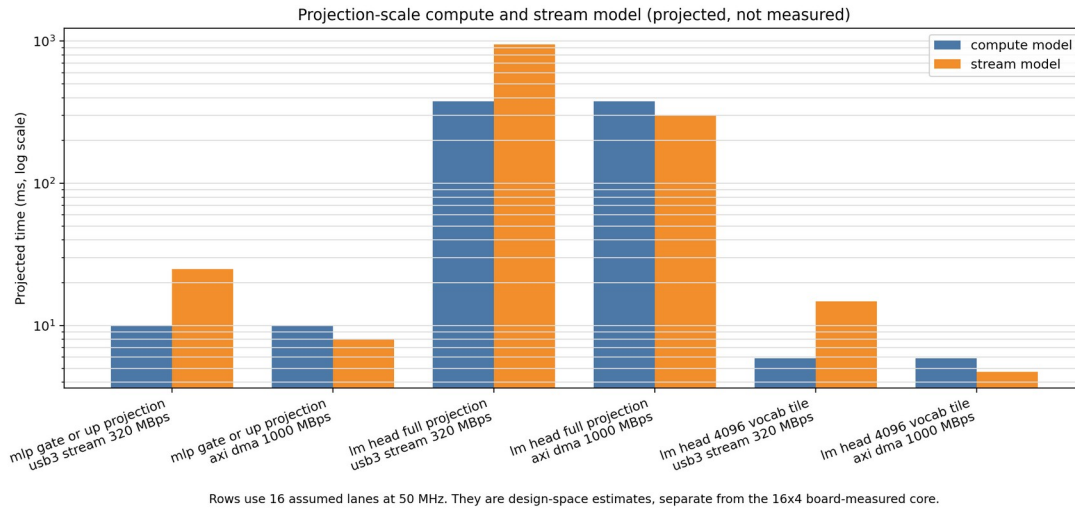


그림 3. Projection-scale roofline 해석

표 8. Projection tile roofline/interface model 요약

component	evidence type	shape	lanes	interface case	compute/stream model
MLP gate/up projection	projected	1152->6912	16	USB3 320 MB/s model case	compute 9.95 ms, stream 24.97 ms
lm_head full projection	projected	1152->262144	16	USB3 320 MB/s model	compute 377.49 ms, stream

component	evidence type	shape	lanes	interface case	compute/stream model
				case	947.00 ms
lm_head tile	projected	1152->4096	16	USB3 320 MB/s model case	projection tile model

DDR2/LPDDR2 다채널 memory는 최신 모바일 LPDDR5보다 높은 단일 채널 성능을 제공한다는 의미가 아니다. 본 연구의 roofline 분석이 시사하는 핵심은 외부 FPGA accelerator가 host interface를 통해 매 token마다 weight를 전송하는 구조가 근본적으로 불리하다는 점이다. 따라서 후속 구조에서 DDR2/LPDDR2를 검토하는 이유는 DRAM 세대의 우열이 아니라, 저비용 custom board에서 weight-resident memory pool을 구성하고 여러 독립 channel 또는 interleaved group을 통해 aggregate bandwidth를 확보할 수 있기 때문이다. 즉, 목표는 Snapdragon 내부 memory hierarchy를 대체하는 것이 아니라, host-FPGA 전송을 반복하지 않는 accelerator-local weight memory를 구성하는 것이다.

FPGA 간 연결은 본질적으로 불가능하거나 반드시 매우 느린 경로가 아니다. 다만 연산용 FPGA와 SRAM-like scratchpad용 FPGA를 custom connector로 연결하려면 pin count, signal integrity, clock-domain crossing, protocol, aggregate bandwidth, board design 난이도가 함께 증가한다. 후속 특기자 산출물에서는 표 5와 표 7의 요구사항을 바탕으로 memory-centric low-bit FPGA NPU를 설계하되, 1.58bit 변환기, SRAM-like scratchpad FPGA, DDR2/LPDDR2 custom board는 각각 별도 검증 대상이 된다.

8. 논의 및 결론

본 연구의 결론은 ONNX Runtime decode trace에서 projection-heavy workload가 뚜렷하게 나타나고, Y700 micrograph에서 해당 projection shape의 온디바이스 provider latency가 millisecond 단위로 관측된다는 것이다. CPUExecutionProvider trace는 병목 후보의 operator share를 보여주고, Y700 micrograph는 그 후보 연산의 Android provider별 절대 실행 비용과 dispatch granularity를 보여준다. KV-cache와 shape-related operator는 long-context decode에서 함께 고려해야 하지만, dense projection을 배제한 QK-only 설계만으로는 본 연구의 관측 결과를 설명하기 어렵다.

Y700 실험은 representative ONNX micrograph benchmark이므로 full decode share를 직접 측정한 결과는 아니다. 그러나 CPU EP와 NNAPI EP에서 projection-scale MatMul/MatMulInteger latency를 확보했고, QAIRT direct DLC 경로로 QNN HTP 실행 가능성과 shape별 차이를 추가로 확인했기 때문에, 기존 host-only profiling을 온디바

이스 실행 비용과 연결하는 근거를 제공한다. 다만 QAIRT direct 결과는 ONNX Runtime QNN EP 결과가 아니며, dynamic weight input을 사용한 float MatMul DLC 결과이다. 따라서 이를 sLLM 전체 추론 가속이나 weight-resident deployment 성능으로 일반화하지 않는다.

FPGA 측면에서는 DE10-Lite 16x4 INT8 MatVec 결과를 통해 최소 연산 단위의 기능 정합성과 cycle-level observability를 확인했다. Roofline/interface model은 projection-scale offload의 핵심 조건이 compute lane 수보다 weight residency, streaming bandwidth, invocation granularity에 있음을 보인다. 따라서 본 논문은 온디바이스 ONNX Runtime 병목 분석과 INT8 MatVec PoC 검증을 바탕으로, 이후 산출물 논문이 사용할 memory-centric low-bit MatVec 구조 요구사항을 정리한 선행 분석으로 둔다.

참고문헌

[1] Shuming Ma, Hongyu Wang, Lingxiao Ma, Lei Wang, Wenhui Wang, Shaohan Huang, Li Dong, Ruiping Wang, Jilong Xue, and Furu Wei. “The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits.” arXiv preprint arXiv:2402.17764, 2024.

[2] Rui-Jie Zhu, Yu Zhang, Steven Abreu, Ethan Sifferman, Tyler Sheaves, Yiqiao Wang, Dustin Richmond, Sumit Bam Shrestha, Peng Zhou, and Jason K. Eshraghian. “Scalable MatMul-free Language Modeling.” arXiv preprint arXiv:2406.02528, 2024.

[3] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. “Efficient Memory Management for Large Language Model Serving with PagedAttention.” arXiv preprint arXiv:2309.06180, 2023.

[4] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness.” arXiv preprint arXiv:2205.14135, 2022.

[5] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. “GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers.” arXiv preprint arXiv:2210.17323, 2022.

[6] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. “SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models.” arXiv preprint arXiv:2211.10438, 2022.

[7] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. “AWQ:

Activation-aware Weight Quantization for LLM Compression and Acceleration.” arXiv preprint arXiv:2306.00978, 2023.

[8] Shulin Zeng, Jun Liu, Guohao Dai, Xinhao Yang, Tianyu Fu, Hongyi Wang, Wenheng Ma, Hanbo Sun, Shiyao Li, Zixiao Huang, Yadong Dai, Jintao Li, Zehao Wang, Ruoyu Zhang, Kairui Wen, Xuefei Ning, and Yu Wang. “FlightLLM: Efficient Large Language Model Inference with a Complete Mapping Flow on FPGAs.” arXiv preprint arXiv:2401.03868, 2024.

[9] Microsoft. “ONNX Runtime Execution Providers” and “Graph Optimizations.” ONNX Runtime Documentation, <https://onnxruntime.ai/docs/execution-providers/> and <https://onnxruntime.ai/docs/performance/model-optimizations/graph-optimizations.html>, accessed 2026-06-29.

[10] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. “Orca: A Distributed Serving System for Transformer-Based Generative Models.” In 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22), pp. 521-538, 2022.

[11] Tri Dao. “FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning.” In 12th International Conference on Learning Representations (ICLR), 2024.

[12] Seongmin Hong, Seungjae Moon, Junsoo Kim, Sungjae Lee, Minsub Kim, Dongsoo Lee, and Joo-Young Kim. “DFX: A Low-latency Multi-FPGA Appliance for Accelerating Transformer-based Text Generation.” In Proceedings of the 55th IEEE/ACM International Symposium on Microarchitecture (MICRO), pp. 616-630, 2022.

[13] Bingbing Li, Santosh Pandey, Haowen Fang, Yanjun Lv, Ji Li, Jieyang Chen, Mimi Xie, Lipeng Wan, Hang Liu, and Caiwen Ding. “FTRANS: Energy-Efficient Acceleration of Transformers using FPGA.” In ACM/IEEE International Symposium on Low Power Electronics and Design (ISLPED), pp. 175-180, 2020.

저자정보

최윤희

한국디지털미디어고등학교

ORCID: [0009-0006-3537-0249](https://orcid.org/0009-0006-3537-0249)